

0-k BFS

Кратчайшие пути с ограниченными целыми весами – C++17

Артёмов Макар Марк Шкут

Сколтех, Алгоритмы 2026

21 мая 2026

Открывающая фраза.

Проект: 0-k BFS для задачи кратчайших путей с ограниченными целыми весами на C++17. Совместная работа с Марком. Цель доклада: объяснить семейство алгоритмов, показать, почему оно обгоняет Дейкстру на графах с малыми целыми весами, и представить наши замеры вплоть до $V = 9 \cdot 10^8$. Длительность: около 15 минут плюс вопросы.

- Кратчайшие пути из одной вершины в графе $G = (V, E, w)$
- Веса принимают значения из $\{0, 1, \dots, k\}$ при небольшом известном k
- Цель: V приближается к 10^9 на одной рабочей станции

Где встречается: планирование путей на сетках, слоистые графы, сведения к 2-SAT, транспортные сети с несколькими уровнями стоимости.

Чётко формулируем задачу.

Нам нужны все расстояния от одной вершины-источника. Особенность в том, что веса — неотрицательные целые числа, ограниченные сверху небольшим k . Это ограничение не искусственное: оно постоянно встречается на сетках, в слоистых DAG, в задачах олимпиадного программирования. Алгоритмически интересно, потому что для маленьких целых весов мы можем превзойти классическую Дейкстру $O((V + E) \log V)$.

Зачем целиться в $V \rightarrow 10^9$: это форсирующее условие дизайна — именно оно потребует неявного графа и 32-битных расстояний, о которых будет дальше.

- `std::priority_queue<pair<Distance,Vertex>`, ленивое decrease-key
- $O((V + E) \log V)$ времени, $O(V + E)$ памяти
- Работает для любых неотрицательных вещественных весов

Множитель $\log V$ – это цена кучи. Если веса малые целые, можем ли мы заменить кучу на что-то более дешёвое?

Определяем Дейкстру как baseline.

Это классический алгоритм, который все знают. Реализация прямая: куча пар (расстояние, вершина) с ленивым decrease-key (повторно кладём, на поп пропускаем устаревшие записи). Множитель $\log V$ платится в каждой успешной релаксации. Вся идея 0-k BFS в том, что при маленьких целых весах куча вообще не нужна.

Пауза для вопроса на слайде – это мотивирует следующий слайд.

Веса 0 или 1. Заменяем очередь с приоритетом на deque:

- ребро веса 0 $\rightarrow$ push_front
- ребро веса 1 $\rightarrow$ push_back

Дек всегда отсортирован по расстоянию, состоит из двух непрерывных слоёв.

Сложность: $O(V + E)$ Память: $O(V)$

Педагогический центр, часть 1.

Идея простая. В любом BFS у нас есть “текущий слой” и “следующий слой”. При весах 0/1 ребро веса 0 оставляет нас в текущем слое (перёд дека, обрабатывается сразу), а ребро веса 1 переводит в следующий (хвост дека). Дек всё время упорядочен по расстоянию и содержит максимум два различных значения.

Важная деталь по сравнению с обычным BFS: вершина может быть открыта несколько раз до того, как её расстояние станет окончательным. Поэтому при каждой релаксации проверяем не “посещена/нет”, а $\text{dist}[v] + w(v,u) < \text{dist}[u]$.

Веса $\{0, 1, \dots, k\}$. Используем $k + 1$ FIFO-очередь, индексируем их по $\text{dist}[v] \bmod (k + 1)$.

- “Активный” бакет cur движется вперёд
- Релаксация кладёт v в бакет $(d_{\text{new}}) \bmod (k + 1)$
- Устаревшие записи пропускаются при поп

Сложность: $O(kV + E)$ Память: $O(k + E)$ на очереди

Педагогический центр, часть 2.

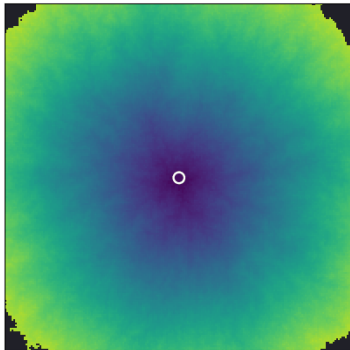
Обобщаем: при весах до k релаксация может продвинуть вершину на расстояние ещё $+k$. Значит, в любой момент живых “слоёв” расстояний не больше $k + 1$. Заводим $k + 1$ FIFO-очереди и индексируем по $\text{dist} \bmod (k + 1)$ – кольцевой буфер бакетов.

Отсюда и название “circular FIFO queues”. Указатель `cur` обходит их; когда текущий бакет опустеет, переходим к следующему. Если вершину переоткрыли с меньшим расстоянием, она попадает в другой бакет, а исходная запись становится устаревшей и пропускается при поп.

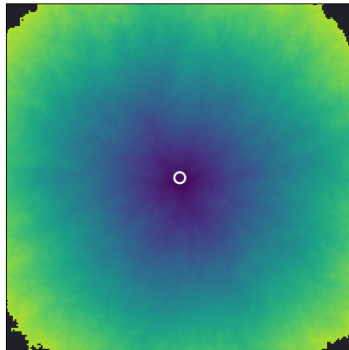
Как это выглядит

Same grid (200x200, $k=8$) · same answer · Dijkstra is 3.32x slower

0-k BFS · 4.00 ms



Dijkstra · 13.30 ms



$d \leq 300.0$ reached 39273 / 40000 (98.2%)

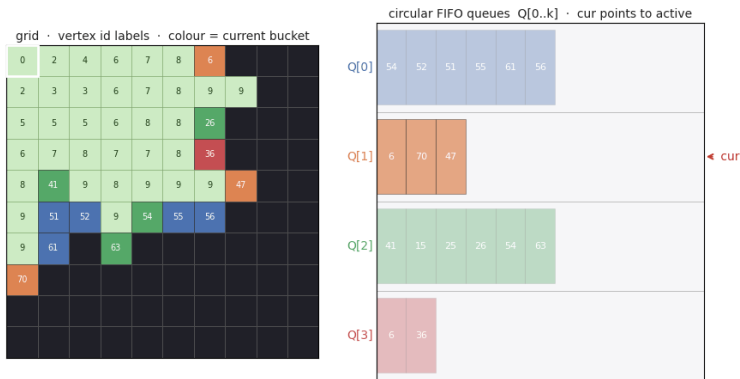
Одна и та же сетка 200×200 , $k = 8$. Один и тот же ответ. 0-k BFS 4.00 мс, Дейкстра 13.30 мс
– в 3.32× медленнее.

Показываем сравнительную анимацию (или этот кадр).

“Оба алгоритма выдают одинаковое поле расстояний. Левая панель завершает волну за 4 миллисекунды, правая за 13.3 – разница вся в том, что каждая релаксация в Дейкстре платит логарифмический фактор кучи. Анимация в репозитории запускает оба алгоритма синхронно, а в заголовках выписаны реальные времена.”

Тезис: визуализация – тоже часть проекта; именно через неё мы сами проверяли алгоритм, прежде чем измерять.

0-k BFS step-by-step · grid 10x10, k=3



step 37: pop $u=17$ from $Q[1]$ (cur=9)

Пошагово на маленькой сетке 10×10 , $k = 3$. Каждая строка справа – одна очередь $Q[0] \dots Q[3]$; **cur** указывает на активный бакет.

Закрепляем структуру данных.

Справа видны реальные четыре очереди $Q[0] - Q[3]$ с их текущим содержимым (числа – идентификаторы вершин). Слева сетка, цвета клеток показывают, в каком баке сейчас лежит вершина. На этом кадре `cur` указывает на $Q[1]$.

Анимация – скрипт `viz/animate_queues.py`, который потребляет пошаговый журнал, выгружаемый инструментированным алгоритмом на C++ (`include/zkbfs/bfs0k_trace.hpp`). Именно эта картинка делает алгоритм обучаемым: вся суть в очередях.

- Явный CSR при $V = 10^9$ стоит ~ 20 ГБ только на рёбра – невозможно
- GridGraph: неявная 4-связная сетка, $O(1)$ на соседа, рёбра не хранятся
- Distance = uint32_t: уменьшает доминирующий массив вдвое (4 ГБ вместо 8 ГБ)
- Допустимо, пока $k \cdot V < 2^{32}$

Защищаем дизайнерские решения, благодаря которым возможен большой запуск. Бэкендов два. Универсальный GraphCSR упирается в потолок около $V \approx 10^8$ из-за памяти. GridGraph неявный: рёбра не храним, вычисляем по запросу. Это плюс 32-битное расстояние – именно так мы фактически прогнали $V = 9 \cdot 10^8$ при 14 ГБ свободной RAM. Честная оговорка: 32-битное расстояние работает не всегда. Требование: $k \cdot V < 2^{32}$. Для наших параметров это выполнено, плюс есть 64-битный fallback через флаг компиляции.

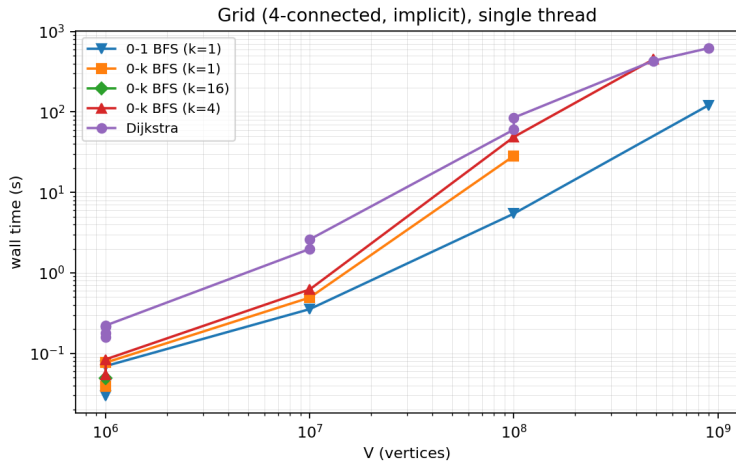
V	k	Дейкстра	0-k BFS	0-1 BFS	ускорение
10^6	1	0.180 с	0.039 с	0.070 с	$4.6\times$
10^7	1	1.98 с	0.493 с	0.355 с	$5.6\times$
10^8	1	60.3 с	28.4 с	5.44 с	$11.1\times$
$4.84 \cdot 10^8$	4	432.9 с	451.4 с	$-0.96\times$ (пам.)	
$9 \cdot 10^8$	1	619.9 с	—	121.2 с	$5.12\times$

Оба алгоритма дают побайтово совпадающие контрольные суммы расстояний на каждом инстансе.

Проходим таблицу сверху вниз.

Уже при $V = 10^6$ получаем ускорение в $4\text{--}5\times$. При $V = 10^8$ и $k = 1$ deque-вариант 0–1 BFS в $11\times$ быстрее Дейкстры. Строка $V \approx 5 \cdot 10^8$ интересна – разрыв схлопывается, оба алгоритма становятся memory-bandwidth bound. Внизу заголовочный результат: на 900 миллионах вершин 0–1 BFS заканчивает за две минуты, Дейкстра – за десять.

Важно: все контрольные суммы совпадают; корректность проверена на 19 различных инстансах в свипе, ноль расхождений.



Разбираем график.

Обе оси логарифмические. Пять серий: Дейкстра вверху, варианты BFS идут параллельно ниже. Постоянный вертикальный зазор в логарифмической шкале соответствует примерно постоянному мультипликативному ускорению. При $V = 10^8$ и выше линии начинают сходиться – входим в memory-bound режим. Самая нижняя линия везде – deque-вариант 0–1 BFS, наиболее чистая реализация в семействе.

- Ускорение уменьшается с ростом k – исчезает при $k = \omega(\log V)$
- При $V \gtrsim 5 \cdot 10^8$ оба алгоритма упираются в пропускную способность памяти
- Двухочередной *bfs_0k* медленнее однодекового *bfs_01* при большом V (аллокатор) – $k + 1$ очередь = $k + 1$ аллокатор
- Требуются неотрицательные целые веса с известным малым k

Честно про границы применимости.

Во-первых: это выигрыш по константе, не по асимптотике. Как только k сравним с $\log V$, Дейкстра не хуже.

Во-вторых: на гигантских V всё упирается в память. Мы видели, как 0– k BFS приходит к паритету с Дейкстрой при $V \approx 5 \cdot 10^8$. Реализация имеет огромное значение: содержать $k + 1$ отдельный аллокатор очередей дороже для кэша и аллокатора, чем один deque – поэтому 0–1 BFS у нас продолжает выигрывать при $V = 9 \cdot 10^8$, а общий 0– k BFS на этом масштабе уже нет.

В-третьих: это не замена Дейкстре в общем случае. Для вещественных весов или неизвестного бакета – использовать Дейкстру.

- Индексная d -арная куча для Дейкстры (честный decrease-key)
- Кусочный кольцевой FIFO для bfs_0k (закрыть разрыв с 0-1 BFS)
- Загрузчик дорожной сети (DIMACS USA, веса квантуем до $k = 8$)
- Стретч: Δ -stepping для параллельного контекста

Куда движемся.

Четыре пункта. Первые два – доводка реализации: настоящая куча с decrease-key как более честный baseline и кусочная очередь, чтобы догнать cache-поведение дека. Третий – выйти из зоны синтетических графов на реальную дорожную сеть. Четвёртый – Δ -stepping, это канонический параллельный родственник 0-k BFS; полезно иметь хотя бы для контекста, даже если параллелизм мы не разгоняем всерьёз.

- Код, отчёт, графики: github.com/artemovmak/EffectiveAlgorithms
- Отчёт по Этапу 1: [stage1_report.pdf](#)

Вопросы?

Заккрытие.

Всё – код, LaTeX-отчёт, анимации, воспроизводимые скрипты сборки – лежит в репозитории на GitHub. Переходим к вопросам. Вероятные вопросы:

В: “Почему 0–1 BFS быстрее 0– k BFS при $k = 1$?” – накладные расходы аллокатора двух `std::queue` против одного `std::deque`.

В: “Как проверяли корректность на запуске с миллиардом вершин?” – контрольные суммы расстояний совпадают с Дейкстрой побайтово; поэлементное сравнение не нужно, сумма уже фиксирует массив.

В: “А параллелизм?” – весь Этап 1 в один поток; Δ -stepping – запланированный стретч-таргет на Неделю 2.